I’m doing a lab on a terminal Blackjack game. Here are the three files. Don’t change anything yet. First, explain how hand_value works and what is wrong with how it handles Aces. Then list any other problems in game.py (round resolution, chips, edge cases).

cards.py

```
import random

RANKS = ["2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"]
SUITS = ["C", "D", "H", "S"]

class Deck:
def __init__(self):
self.cards = [(rank, suit) for suit in SUITS for rank in RANKS]
random.shuffle(self.cards)

def draw(self):
return self.cards.pop() if self.cards else None

def hand_value(hand):

value = sum(11 if rank == "A" else 10 if rank in {"J", "Q", "K"} else int(rank)
for rank, _ in hand)
return value
```

game.py

```
from cards import Deck, hand_value

class Blackjack:
def __init__(self):
self.chips = 100

def show(self, player, dealer, hide=True):
shown_dealer = ["??"] if hide else [f"{r}{s}" for r, s in dealer]
print("Dealer:", " ".join(shown_dealer))
print("Player:", " ".join(f"{r}{s}" for r, s in player),
"=", hand_value(player))

def round(self):
deck = Deck()
player = [deck.draw(), deck.draw()]
```

```
dealer = [deck.draw(), deck.draw()]
self.show(player, dealer)

while hand_value(player) < 21:
key = input("[h]it [s]tand [q]uit: ").strip().lower()
if key == "q":
return False
if key == "s":
break
if key == "h":
player.append(deck.draw())
self.show(player, dealer)
if hand_value(player) > 21:
print("Bust.")
return True
while hand_value(dealer) < 17:
dealer.append(deck.draw())

self.show(player, dealer, hide=False)
pv, dv = hand_value(player), hand_value(dealer)
if dv > 21 or pv > dv:
self.chips += 10
print("Player wins.")
elif pv < dv:
self.chips -= 10
print("Dealer wins.")
else:
print("Push.")
return True

def run(self):
print("Blackjack ΓÇö starting chips:", self.chips)
while self.chips > 0:
if not self.round():
return
if input("Play again? [y/n]: ").strip().lower() != "y":
return
```

main.py

```
from game import Blackjack


if __name__ == "__main__":
    Blackjack().run()
```

Nothing changed, as requested. Here's the analysis.

How hand_value works

It maps each card to a number and sums them: number cards count at face value (int(rank)), J/Q/K count 10, and every Ace counts 11. It returns that sum.

What's wrong with Aces

Aces are hard-coded as 11 and never downgraded. Blackjack requires an Ace to be 1 or 11, whichever is best for the hand. The consequences:

- **False busts.** A hand of A + 5 + 9 is really 15 (Ace as 1) but this function returns 25, so the player is told "Bust."
- **Two Aces are always 22.** The opening hand A + A is a bust on the deal. The correct value is 12.
- **Dealer is affected too.** The dealer's while hand_value(dealer) < 17 loop stops incorrectly, and a dealer with A + 6 (soft 17) is handled by the wrong numbers. Dealer hands can show as busted when they aren't, which hands the player free wins.
- **The fix is conceptually simple:** sum with Aces as 11, then while the total exceeds 21 and there's an Ace still counted as 11, subtract 10. This also lets you detect "soft" hands (useful for the dealer-hits-soft-17 rule, if you want one).

Other problems in game.py

Round resolution

1. **A bust never costs chips.** When the player busts, return True fires with no self.chips -= 10. The player can't lose by busting, which is a major bug.
2. **No blackjack detection.** A natural 21 on the deal isn't recognized. There's no 3:2 payout, and the loop while hand_value(player) < 21 just skips the player's turn silently. The dealer's natural isn't checked either, and neither side can push on a mutual natural.
3. **Dealer plays even when it doesn't need to.** If the player busts, you correctly return early, but there's no early-exit logic for a player's natural either.

4. **Dealer's second card isn't hidden properly.** show replaces the whole dealer hand with a single "??", so the player sees no upcard. Normally one card is visible and one is hidden.
5. **Result display is mixed with logic.** Win/loss messages and chip changes are interleaved in round, which makes it hard to test.

Chips and betting

6. **Fixed bet of 10, hard-coded and unvalidated.** There's no betting prompt, no way to bet more or less, and no check that the player can afford it. With e.g. 5 chips left, a loss takes you to -5.
7. **No double down, split, or insurance.** Fine for a basic lab, but worth stating as a scope limit.
8. **run never reports final chips or why the game ended.** When chips hit 0 the loop just exits silently.

Edge cases and robustness

9. **q mid-round forfeits nothing.** Returning False quits with no chip change, so the player can escape a bad hand for free.
10. **Invalid input is silently ignored.** Any key other than h/s/q just re-prompts with no message. Not a bug exactly, but unfriendly.
11. **input() can raise EOFError/KeyboardInterrupt.** Piped input or Ctrl-D/Ctrl-C crashes with a traceback.
12. **deck.draw() can return None.** It does for an empty deck. With a fresh 52-card deck per round it can't realistically happen, but if it did, hand_value would crash unpacking None.
13. **A fresh Deck() every round.** This means no card counting and no shuffle-point logic. Acceptable, but it's a design choice that should be intentional.
14. **Stand and 21 are handled inconsistently.** With exactly 21, the loop exits without a prompt (good), but it's by accident of < 21, not a deliberate "auto-stand."

Encoding

15. **Mojibake in run.** "Blackjack ΓÇö starting chips" is a UTF-8 em dash (—) that was mis-decoded as Windows-1252/CP437 and re-saved. Replace it with a real — or just a hyphen, and make sure the file is saved as UTF-8.

Priority order if you fix things

Most impactful first: Ace handling (#1 above in hand_value), bust not deducting chips, blackjack/natural handling, then betting validation, then the rest. Tell me which of these you'd like me to tackle, and whether the lab expects a minimal fix or a fuller rewrite.

Fix only the hand_value function in cards.py. An Ace should count as 11 unless that would bust the hand, otherwise 1. It must handle multiple Aces. Expected: A+9 = 20, A+9+5 = 15, A+A+9 = 21. Show only the changed function and explain it.

```
python

def hand_value(hand):
    value = 0
    aces = 0
    for rank, _ in hand:
        if rank == "A":
            value += 11
            aces += 1
        elif rank in {"J", "Q", "K"}:
            value += 10
        else:
            value += int(rank)


    while value > 21 and aces:
        value -= 10
        aces -= 1


    return value
```

How it works

1. **First pass:** every card is added up with Aces counted as 11, and a separate counter (aces) tracks how many Aces are currently being counted as 11. Face cards are 10 and number cards are their face value, same as before.
2. **Downgrade pass:** while the total is over 21 and at least one Ace is still counted as 11, subtract 10. That turns that Ace from 11 into 1. Each downgrade uses up one Ace from the counter, so the loop stops when the hand is no longer bust or there are no Aces left to downgrade.

Because Aces are downgraded one at a time and only as needed, multiple Aces work correctly: at most one Ace ever stays at 11 (two at 11 would already be 22), and the rest drop to 1.

Checking your expected cases

Hand	First pass	Over 21?	After downgrades	Result
A + 9	20	No	none needed	**20**
A + 9 + 5	25	Yes	one Ace: 25 - 10	**15**
A + A + 9	31	Yes	one Ace: 21, now not over	**21**

A few extra cases that also come out right: A + A = 12 (22 - 10), A + K = 21 (a natural, left as 21 since it isn't over), and A + A + A + 8 = 21 (41 → 31 → 21, two Aces downgraded, one stays 11).

The function still takes the same input and returns an int, so game.py needs no changes to use it.

Task 2: Complete the round resolution in game.py. Handle natural blackjack (21 on the first two cards), player bust, dealer bust, dealer drawing until 17 or more, and pushes consistently. Prevent any action after the round has ended. Keep changes small and don't rewrite the whole project. Show the changed code and explain it.

Only round changes, plus a constant, a small helper, and a finish method. show, run, cards.py and main.py are untouched.

python

```
from cards import Deck, hand_value


BET = 10



def is_blackjack(hand):
    """A natural: exactly two cards totalling 21."""
    return len(hand) == 2 and hand_value(hand) == 21
```

```
class Blackjack:
    def __init__(self):
        self.chips = 100

    # show(...) unchanged

    def finish(self, delta, message):
        """Single exit point for a finished round: apply chips, report, end."""
        self.chips += delta
        print(f"{message} Chips: {self.chips}")
        return True

    def round(self):
        deck = Deck()
        player = [deck.draw(), deck.draw()]
        dealer = [deck.draw(), deck.draw()]
        self.show(player, dealer)

        # Naturals are settled immediately, before the player can act.
        p_nat, d_nat = is_blackjack(player), is_blackjack(dealer)
        if p_nat or d_nat:
            self.show(player, dealer, hide=False)
            if p_nat and d_nat:
                return self.finish(0, "Both have blackjack. Push.")
            if p_nat:
                return self.finish(BET * 3 // 2, "Blackjack! Player wins 3:2.")
```

```
    return self.finish(-BET, "Dealer has blackjack. Dealer wins.")

# Player turn
while hand_value(player) < 21:
    key = input("[h]it [s]tand [q]uit: ").strip().lower()
    if key == "q":
        return False
    if key == "s":
        break
    if key == "h":
        player.append(deck.draw())
        self.show(player, dealer)
        if hand_value(player) > 21:
            return self.finish(-BET, "Bust. Dealer wins.")

# Dealer turn: draw until 17 or more
while hand_value(dealer) < 17:
    dealer.append(deck.draw())

self.show(player, dealer, hide=False)
pv, dv = hand_value(player), hand_value(dealer)
if dv > 21:
    return self.finish(BET, "Dealer busts. Player wins.")
if pv > dv:
    return self.finish(BET, "Player wins.")
if pv < dv:
```

```
            return self.finish(-BET, "Dealer wins.")
        return self.finish(0, "Push.")


    # run(...) unchanged
```

What changed and why

One exit path for every outcome. finish(delta, message) applies the chip change, prints the result with the new balance, and returns True. Every ending (natural, bust, dealer bust, win, loss, push) goes through it. That keeps the chip logic consistent, so the old bug where a player bust cost nothing can't recur.

Naturals are checked first. is_blackjack requires exactly two cards totalling 21, so A+K counts but a three-card 21 doesn't. If either side has one, the dealer's hand is revealed and the round ends before any input:

- Both naturals: push, no chips move.
- Player only: pays 3:2, which is +15 on a bet of 10.
- Dealer only: the player loses the bet.

Player bust. A hit that takes the total over 21 now returns finish(-BET, ...) immediately. The dealer never plays against a busted player.

Dealer turn. The dealer draws while under 17 and stands on any 17 or more. With the fixed hand_value, a soft 17 (A+6) counts as 17, so the dealer stands on it.

Comparison. The dealer bust is checked first, then higher, lower, and equal. Equal is a push with delta = 0, printed through the same path as everything else.

No action after the round ends. Every outcome is a return, and the code after a return can't run. After a bust there's no dealer draw or comparison. After a natural there's no player prompt. The input loop also can't continue at 21, since while hand_value(player) < 21 ends it. A player who hits to exactly 21 therefore auto-stands, and a two-card 21 never reaches the loop because naturals are settled earlier.

Left alone

These are still open from the earlier list, in case your lab covers them:

- q mid-round still returns False without costing chips.
- There's no affordability check, so chips can go below 0 if you have less than 10 left.

- The dealer’s hand is still fully hidden as ?? rather than showing one upcard.
- The mojibake in the run title string is still there.

```
D:\SE\18_blackjack>python main.py
Traceback (most recent call last):
File "D:\SE\18_blackjack\main.py", line 4, in <module>
Blackjack().run()
^^^^^^^^^^^^^^^^
AttributeError: 'Blackjack' object has no attribute 'run'

D:\SE\18_blackjack>
```

That’s my fault. In the last reply I wrote # show(...) unchanged and # run(...) unchanged as placeholder comments, and if you pasted the snippet over game.py, those methods were deleted. show would have been the next thing to fail.

Here is the complete game.py with show and run restored exactly as in your original:

python

```
from cards import Deck, hand_value


BET = 10



def is_blackjack(hand):
    """A natural: exactly two cards totalling 21."""
    return len(hand) == 2 and hand_value(hand) == 21



class Blackjack:
    def __init__(self):
        self.chips = 100
```

```
def show(self, player, dealer, hide=True):
    shown_dealer = ["??"] if hide else [f"{r}{s}" for r, s in dealer]
    print("Dealer:", " ".join(shown_dealer))
    print("Player:", " ".join(f"{r}{s}" for r, s in player),
          "=", hand_value(player))

def finish(self, delta, message):
    """Single exit point for a finished round: apply chips, report, end."""
    self.chips += delta
    print(f"{message} Chips: {self.chips}")
    return True

def round(self):
    deck = Deck()
    player = [deck.draw(), deck.draw()]
    dealer = [deck.draw(), deck.draw()]
    self.show(player, dealer)

    # Naturals are settled immediately, before the player can act.
    p_nat, d_nat = is_blackjack(player), is_blackjack(dealer)
    if p_nat or d_nat:
        self.show(player, dealer, hide=False)
        if p_nat and d_nat:
            return self.finish(0, "Both have blackjack. Push.")
        if p_nat:
            return self.finish(BET * 3 // 2, "Blackjack! Player wins 3:2.")
```

```
    return self.finish(-BET, "Dealer has blackjack. Dealer wins.")

# Player turn
while hand_value(player) < 21:
    key = input("[h]it [s]tand [q]uit: ").strip().lower()
    if key == "q":
        return False
    if key == "s":
        break
    if key == "h":
        player.append(deck.draw())
        self.show(player, dealer)
        if hand_value(player) > 21:
            return self.finish(-BET, "Bust. Dealer wins.")

# Dealer turn: draw until 17 or more
while hand_value(dealer) < 17:
    dealer.append(deck.draw())

self.show(player, dealer, hide=False)
pv, dv = hand_value(player), hand_value(dealer)
if dv > 21:
    return self.finish(BET, "Dealer busts. Player wins.")
if pv > dv:
    return self.finish(BET, "Player wins.")
if pv < dv:
```

```
            return self.finish(-BET, "Dealer wins.")
        return self.finish(0, "Push.")


    def run(self):
        print("Blackjack ΓÇö starting chips:", self.chips)
        while self.chips > 0:
            if not self.round():
                return
            if input("Play again? [y/n]: ").strip().lower() != "y":
                return
```

Only BET, is_blackjack, finish and round differ from your original. show and run are identical, including the garbled ΓÇö in the title string, which I left alone since you asked for small changes. If it prints oddly in your terminal, replacing it with a plain - is a safe fix.

Going forward, if I mark something as “unchanged” in a snippet, treat that as an instruction to keep your existing code there, not to paste over it.

How can I test a natural blackjack without waiting for lucky cards? Give me a small command I can run in cmd using my existing code.

Run this from D:\SE\18_blackjack:

python -c "import random; random.shuffle=lambda x: None; from game import Blackjack; Blackjack().round()"

It turns off shuffling, so the deck stays in the order Deck.__init__ builds it. draw() pops from the end, so the player gets A♠ and K♠ (a natural) and the dealer gets Q♠ and J♠ (20). You should see both hands revealed, then Blackjack! Player wins 3:2. Chips: 115, with no prompt.

Testing other outcomes

This version lets you choose the exact cards. The cards are dealt in the order player, player, dealer, dealer:

python -c "import game; order=[('9','S'),('7','H'),('A','D'),('K','C')]; game.Deck=lambda: type('D',(),{'cards':order[::-1],'draw':lambda s:s.cards.pop()})(); game.Blackjack().round()"

As written, it tests a dealer natural: the player has 9 and 7, the dealer has A and K, and you should see Dealer has blackjack. Dealer wins. Chips: 90. Change the four cards in order to try other cases:

Case	order (p1, p2, d1, d2)	Expected
Both naturals (push)	('A','S'),('K','S'),('A','H'),('K','H')	Both have blackjack. Push. Chips: 100
A+A+9 isn’t a natural	('A','S'),('A','H'),('9','D'),('5','C')	Prompts you (hand is 12)

For anything that isn’t a natural, the game asks for input as usual, so you can also test hit, stand and bust. After a hit, the deck runs out of your chosen cards and draw() returns None, which crashes hand_value. If you want to test those paths, add enough extra cards to the end of order.

Blackjack().round() skips run(), so chips start at 100 each time. Nothing in your files is modified, because the patching only exists inside that one command.

Task 3: Make chips persist across rounds and add a simple wager prompt with validation (positive integer, not more than current chips). Each round must settle exactly once. Show chips at the start of each round. Keep all state in memory and add no files or new dependencies.

Chips already live on the Blackjack instance, so they carry across rounds. What was missing was the wager, the display, and a guarantee that a round settles only once. Here is the complete game.py, with show and run included so nothing gets deleted when you paste it:

python

```
from cards import Deck, hand_value



def is_blackjack(hand):
    """A natural: exactly two cards totalling 21."""
    return len(hand) == 2 and hand_value(hand) == 21



class Blackjack:
```

```
def __init__(self):
    self.chips = 100
    self.bet = 0
    self.settled = True  # no round in progress yet

def show(self, player, dealer, hide=True):
    shown_dealer = ["??"] if hide else [f"{r}{s}" for r, s in dealer]
    print("Dealer:", " ".join(shown_dealer))
    print("Player:", " ".join(f"{r}{s}" for r, s in player),
          "=", hand_value(player))

def ask_bet(self):
    """Prompt until a whole number from 1 to self.chips. None = quit."""
    while True:
        raw = input(f"Bet (1-{self.chips}, q to quit): ").strip().lower()
        if raw == "q":
            return None
        if raw.isascii() and raw.isdigit():
            bet = int(raw)
            if 1 <= bet <= self.chips:
                return bet
        print(f"Enter a whole number from 1 to {self.chips}.")

def finish(self, delta, message):
    """Single exit point for a finished round: settle once, report, end."""
    if self.settled:
```

```
        raise RuntimeError("Round already settled")
    self.settled = True
    self.chips += delta
    print(f"{message} Chips: {self.chips}")
    return True

def round(self):
    print(f"\nChips: {self.chips}")
    bet = self.ask_bet()
    if bet is None:
        return False
    self.bet = bet
    self.settled = False

    deck = Deck()
    player = [deck.draw(), deck.draw()]
    dealer = [deck.draw(), deck.draw()]
    self.show(player, dealer)

    # Naturals are settled immediately, before the player can act.
    p_nat, d_nat = is_blackjack(player), is_blackjack(dealer)
    if p_nat or d_nat:
        self.show(player, dealer, hide=False)
        if p_nat and d_nat:
            return self.finish(0, "Both have blackjack. Push.")
        if p_nat:
```

```
        return self.finish(self.bet * 3 // 2, "Blackjack! Player wins 3:2.")
    return self.finish(-self.bet, "Dealer has blackjack. Dealer wins.")

# Player turn
while hand_value(player) < 21:
    key = input("[h]it [s]tand [q]uit: ").strip().lower()
    if key == "q":
        return False
    if key == "s":
        break
    if key == "h":
        player.append(deck.draw())
        self.show(player, dealer)
        if hand_value(player) > 21:
            return self.finish(-self.bet, "Bust. Dealer wins.")

# Dealer turn: draw until 17 or more
while hand_value(dealer) < 17:
    dealer.append(deck.draw())

self.show(player, dealer, hide=False)
pv, dv = hand_value(player), hand_value(dealer)
if dv > 21:
    return self.finish(self.bet, "Dealer busts. Player wins.")
if pv > dv:
    return self.finish(self.bet, "Player wins.")
```

```
        if pv < dv:
            return self.finish(-self.bet, "Dealer wins.")
        return self.finish(0, "Push.")


    def run(self):
        print("Blackjack ΓÇö starting chips:", self.chips)
        while self.chips > 0:
            if not self.round():
                return
            if self.chips <= 0:
                print("Out of chips. Game over.")
                return
            if input("Play again? [y/n]: ").strip().lower() != "y":
                return
```

What changed

Persistence. self.chips is set once in __init__ and only changed inside finish. Each round builds a new Deck, but the same Blackjack object lives for the whole session, so the balance carries over. Nothing is written to disk.

Chips at round start. round prints Chips: N before anything else, so you see the balance before every bet.

Wager prompt (ask_bet). It loops until the input is a whole number from 1 to your current chips. Letters, blanks, 0, negatives, decimals and anything above your balance get a message and a re-prompt. q at the wager quits cleanly. The isascii() check stops things like ² that pass isdigit() but fail int(). isdigit() also rejects -5 and 1.5, so no separate sign check is needed.

Bet replaces the constant. BET = 10 is gone. The bet is stored as self.bet and every payout uses it, so a player can never lose more than they staked, and a bet is always affordable.

Settle exactly once. settled is set to False when a round starts and finish sets it to True. If any code path called finish twice in one round, it would raise RuntimeError instead of silently double-paying. Every ending already returns through finish, so this is a safety net.

Out of chips. When a round drops you to 0, run now prints a message and stops, instead of silently skipping the “Play again?” prompt.

Things to know

- **3:2 on odd bets rounds down.** A bet of 5 pays 7 (5 * 3 // 2), not 7.5, since chips are integers.
- **Quitting mid-hand is still free.** q during the player’s turn returns False with no chip change, so the round never settles. I left it alone since you didn’t ask, but if your lab says every started round must settle, the fix is to make q forfeit with return self.finish(-self.bet, "Forfeit.").

Quick validation test (cmd)

(echo abc & echo 0 & echo -5 & echo 500 & echo 10 & echo s & echo n) | python main.py

You should see four rejections, then a valid bet of 10. If the first deal is a natural, the round ends instantly and the s answers the “Play again?” prompt, which still exits cleanly.

Task 4: Add short messages for each card drawn (for example “Player draws 9H”, “Dealer draws KD”) and for each round outcome, including the chip change. Invalid commands (anything other than h, s or q) must show a message and must not change the bankroll. Also handle an empty deck without crashing.

Here is the complete game.py, with show and run included so nothing gets deleted when you paste it. cards.py and main.py don’t change.

python

```
from cards import Deck, hand_value




class EmptyDeck(Exception):
    """Raised when a card is needed but the deck has none left."""




def is_blackjack(hand):
    """A natural: exactly two cards totalling 21."""
```

```
    return len(hand) == 2 and hand_value(hand) == 21



def draw_card(deck, hand, who, hidden=False):
    """Draw one card into hand and announce it. Raises EmptyDeck if none left."""
    card = deck.draw()
    if card is None:
        raise EmptyDeck
    hand.append(card)
    if hidden:
        print(f"{who} draws a card")
    else:
        print(f"{who} draws {card[0]}{card[1]}")



class Blackjack:
    def __init__(self):
        self.chips = 100
        self.bet = 0
        self.settled = True  # no round in progress yet

    def show(self, player, dealer, hide=True):
        shown_dealer = ["??"] if hide else [f"{r}{s}" for r, s in dealer]
        print("Dealer:", " ".join(shown_dealer))
        print("Player:", " ".join(f"{r}{s}" for r, s in player),
              "=", hand_value(player))
```

```
def ask_bet(self):
    """Prompt until a whole number from 1 to self.chips. None = quit."""
    while True:
        raw = input(f"Bet (1-{self.chips}, q to quit): ").strip().lower()
        if raw == "q":
            return None
        if raw.isascii() and raw.isdigit():
            bet = int(raw)
            if 1 <= bet <= self.chips:
                return bet
        print(f"Enter a whole number from 1 to {self.chips}.")


def finish(self, delta, message):
    """Single exit point for a finished round: settle once, report, end."""
    if self.settled:
        raise RuntimeError("Round already settled")
    self.settled = True
    self.chips += delta
    change = f"{delta:+d} chips" if delta else "no chip change"
    print(f"{message} ({change}) Chips: {self.chips}")
    return True


def round(self):
    print(f"\nChips: {self.chips}")
    bet = self.ask_bet()
```

```
    if bet is None:
        return False
    self.bet = bet
    self.settled = False
    try:
        return self._play()
    except EmptyDeck:
        return self.finish(0, "Deck ran out of cards. Round void, bet returned.")


def _play(self):
    deck = Deck()
    player, dealer = [], []
    draw_card(deck, player, "Player")
    draw_card(deck, player, "Player")
    draw_card(deck, dealer, "Dealer", hidden=True)
    draw_card(deck, dealer, "Dealer", hidden=True)
    self.show(player, dealer)

    # Naturals are settled immediately, before the player can act.
    p_nat, d_nat = is_blackjack(player), is_blackjack(dealer)
    if p_nat or d_nat:
        self.show(player, dealer, hide=False)
        if p_nat and d_nat:
            return self.finish(0, "Both have blackjack. Push.")
        if p_nat:
            return self.finish(self.bet * 3 // 2, "Blackjack! Player wins 3:2.")
```

```
    return self.finish(-self.bet, "Dealer has blackjack. Dealer wins.")

# Player turn
while hand_value(player) < 21:
    key = input("[h]it [s]tand [q]uit: ").strip().lower()
    if key == "q":
        return False
    if key == "s":
        break
    if key == "h":
        draw_card(deck, player, "Player")
        self.show(player, dealer)
        if hand_value(player) > 21:
            return self.finish(-self.bet, "Bust. Dealer wins.")
    else:
        print(f"Invalid command '{key}'. Use h, s or q.")

# Dealer turn: draw until 17 or more
while hand_value(dealer) < 17:
    draw_card(deck, dealer, "Dealer")

self.show(player, dealer, hide=False)
pv, dv = hand_value(player), hand_value(dealer)
if dv > 21:
    return self.finish(self.bet, "Dealer busts. Player wins.")
if pv > dv:
```

```
        return self.finish(self.bet, "Player wins.")
    if pv < dv:
        return self.finish(-self.bet, "Dealer wins.")
    return self.finish(0, "Push.")


def run(self):
    print("Blackjack ΓÇö starting chips:", self.chips)
    while self.chips > 0:
        if not self.round():
            return
        if self.chips <= 0:
            print("Out of chips. Game over.")
            return
        if input("Play again? [y/n]: ").strip().lower() != "y":
            return
```

What changed

Draw messages. All dealing now goes through draw_card, which draws, adds the card to the hand and prints Player draws 9H or Dealer draws KD. The player's cards (opening two and every hit) and every dealer hit are announced. The dealer's two opening cards print Dealer draws a card instead, so the hidden hand isn't leaked. They're revealed at the end by show(..., hide=False). Cards are still dealt in the order player, player, dealer, dealer, so your earlier test commands behave the same.

Outcome messages with chip change. finish now appends the change, for example Player wins. (+10 chips) Chips: 110, Bust. Dealer wins. (-25 chips) Chips: 75, or Push. (no chip change) Chips: 100. Since every ending goes through finish, none can be missed.

Invalid commands. The player loop now has an else branch that prints Invalid command 'x'. Use h, s or q. and loops back to the prompt. That path never touches finish or self.chips, so the bankroll can't change. An empty Enter is treated the same way.

Empty deck. Deck.draw() returns None when empty, so draw_card checks for that and raises EmptyDeck. round catches it and settles the round as void: bet returned, no chips moved, with a message. This goes through finish(0, ...), so the round still settles exactly once. To keep round small, the game logic moved into _play, and round just handles the bet, the settle flag and the EmptyDeck catch.

Tests (cmd, from D:\SE\18_blackjack)

Invalid commands (the bankroll should stay at 100 until the final result):

(echo 10 & echo x & echo hello & echo s) | python -c "import game; order=[('9','S'),('7','H'),('10','D'),('8','C')]; game.Deck=lambda: type('D',(),{'cards':order[::-1],'draw':lambda s:s.cards.pop() if s.cards else None})(); game.Blackjack().round()"

You should see two Invalid command lines, then the player stands on 16 against the dealer's 18 and loses 10.

Empty deck (the dealer needs a card but none are left):

(echo 10 & echo s) | python -c "import game; order=[('9','S'),('7','H'),('10','D'),('2','C')]; game.Deck=lambda: type('D',(),{'cards':order[::-1],'draw':lambda s:s.cards.pop() if s.cards else None})(); game.Blackjack().round()"

You should see Deck ran out of cards. Round void, bet returned. (no chip change) Chips: 100. The fake deck here returns None when empty, like the real one. The fake deck in my earlier test command would have raised IndexError instead, so use these versions.

Give me a small command I can run in cmd to test the empty-deck case. It should show that the game doesn't crash when the deck runs out.

Run this from D:\SE\18_blackjack:

(echo 10 & echo s) | python -c "import game; order=[('9','S'),('7','H'),('10','D'),('2','C')]; game.Deck=lambda: type('D',(),{'cards':order[::-1],'draw':lambda s:s.cards.pop() if s.cards else None})(); game.Blackjack().round()"

It replaces the deck with exactly four cards, dealt as player 9S, player 7H, dealer 10D, dealer 2C. The player stands on 16, and the dealer (12) has to draw but the deck is empty. Like the real Deck.draw(), the fake one returns None when empty. Nothing in your files is modified.

You should see output like this (the bet and s are piped in, so they don't echo):

Chips: 100

Bet (1-100, q to quit): Player draws 9S

Player draws 7H

Dealer draws a card

Dealer draws a card

Dealer: ??

Player: 9S 7H = 16

[h]it [s]tand [q]uit: Deck ran out of cards. Round void, bet returned. (no chip change) Chips: 100

There’s no traceback, and the chips are still 100.

To test the deck running out on the player’s hit instead, change the dealer’s cards to ('10','D'),('8','C') and replace echo s with echo h. The player hits on 16, no card is left, and you get the same “Round void” message.

This is the read-only copy that you shared of your chat.